8 - Tipos Abstratos de Dados e Subprogramas

Na camada de programação, passamos da concretude da linguagem de máquina para a linguagem assembly, para o pseudocódigo e para algoritmos. Em seguida, passamos de algoritmos que usam variáveis simples para algoritmos que usam arrays.

Agora, damos um passo à frente na abstração e falamos sobre contêineres abstratos: estruturas compostas para as quais não conhecemos a implementação. Nos círculos da computação, esses contêineres abstratos são chamados de **tipos abstratos de dados**. Conhecemos suas propriedades e operações e entendemos quais tipos de valores eles podem conter, mas não temos informações sobre sua estrutura interna ou implementação. Ou seja, sabemos quais são as operações e o que elas fazem, mas não sabemos como as operações são implementadas.

O projeto de algoritmo que temos usado é um modelo top-down, no qual dividimos uma tarefa em partes menores. Concluímos este capítulo com mais informações sobre instruções de subprogramas, que são tanto uma maneira de fazer o código espelhar o projeto quanto a maneira como algoritmos e subalgoritmos se comunicam.

OBJETIVOS

Após estudar este capítulo, você deverá ser capaz de:

- distinguir entre uma visualização baseada em array e uma visualização encadeada.
- distinguir entre um array e uma lista.
- distinguir entre uma lista não ordenada e uma lista ordenada.
- distinguir entre o comportamento de uma pilha e o comportamento de uma fila.
- distinguir entre uma árvore binária e uma árvore de busca binária.
- desenhar a árvore de busca binária que é construída a partir da inserção de uma série de itens.
- entender a diferença entre uma árvore e um grafo.
- explicar o conceito de subprogramas e parâmetros e distinguir entre parâmetros de valor e de referência.

8.1 O que é um Tipo Abstrato de Dado?

Um Tipo Abstrato de Dado (TAD) é um contêiner cujas propriedades (dados e operações) são especificadas independentemente de qualquer implementação específica. Lembre-se de que o objetivo no projeto é reduzir a complexidade por meio da abstração. Se pudermos definir estruturas úteis e as operações que manipulam essas estruturas no nível lógico, podemos usá-las como se existissem quando precisarmos delas em nossos projetos.

* Tipo de dado abstrato (TAD): Um contêiner cujas propriedades (dados e operações) são especificadas independentemente de qualquer implementação específica.

Para contextualizar o conceito de TAD, precisamos analisar como visualizamos os dados. Na computação, visualizamos os dados sob três perspectivas: o nível da aplicação, o nível lógico e o nível da implementação.

O nível de aplicação (ou usuário) é a visão dos dados dentro de um problema específico. O nível lógico (ou abstrato) é uma visão abstrata dos valores de dados (o domínio) e das operações que os manipulam. O nível de implementação é uma representação específica da estrutura que contém os itens de dados e a codificação das operações em uma linguagem de programação. Essa visão vê as propriedades representadas como campos de dados e subprogramas específicos. Ela se preocupa com estruturas de dados, a implementação de um campo de dados composto em um TAD.

Os TAD que examinamos neste capítulo são aqueles que a história e a experiência têm mostrado que surgem repetidamente em problemas do mundo real. Esses TADs são contêineres nos quais os itens de dados são armazenados e cada um exibe comportamentos específicos. Eles são chamados de contêineres porque seu único propósito é armazenar outros objetos.

* Estrutura de dados: A implementação de um campo de dados composto em um tipo de dado abstrato.

* Contêineres Objetos: cuja função é armazenar e manipular outros objetos.

8.2 Pilhas (Stacks)

Pilhas e filas são estruturas compostas abstratas que frequentemente são consideradas um par – como manteiga de amendoim e geleia, ou maternidade e torta de maçã. O motivo disso deve ser mais por razões históricas do que por qualquer outra coisa, pois essas duas estruturas têm comportamentos bastante diferentes.

Uma pilha é uma estrutura composta abstrata na qual os acessos são feitos em apenas uma extremidade. Você pode inserir um item como o primeiro e pode remover o primeiro. Este projeto modela muitas coisas na vida real. Contadores chamam isso de LIFO – Last-In Fisrt-Out (que significa "último a entrar, primeiro a sair"). Uma pilha de pratos em um restaurante tem esta propriedade: podemos pegar apenas o prato de cima. Quando fazemos isso, o prato logo abaixo sobe para o topo, i.e., passa a ser o primeiro, para que a próxima pessoa possa pegá-lo. Enlatados na prateleira de um supermercado exibem essa propriedade. Quando pegamos a primeira lata de uma fileira, estamos pegando a última (ou a mais recente) lata colocada naquela fileira.

Outra maneira de declarar o comportamento de acesso de uma pilha é dizer que o item removido é o item que está na pilha há menos tempo. Visualizar uma pilha dessa perspectiva é mais abstrato. A operação de inserção não tem restrições; todo o comportamento LIFO é especificado por meio da operação de remoção.

A imagem mental dos pratos do restaurante deixou uma marca nos nomes tradicionais usados para as operações de inserção e remoção. Adicionar um item à pilha é chamado de **Push** (ou Empilhar); e remover um item é chamado de **Pop** (ou Desempilhar). Nós empilhamos um item na pilha e desempilhamos um item da pilha. Uma pilha não tem a propriedade de comprimento, portanto, não há operação que retorne o número de itens na pilha. No entanto, precisamos de operações que determinem se uma pilha está Vazia, porque tentar retirar um item quando a pilha está vazia é um erro.

Aqui está um algoritmo que lê números e os imprime em ordem reversa usando uma pilha. Não colorimos as operações de pilha em vermelho porque elas já foram implementadas por outra pessoa; elas são nossas para uso. Como a quantidade de dados não é relevante para nossa discussão, deixamos a descrição sem detalhamento aqui e nos algoritmos a seguir.

```
while (mais dados)
   read valor
   Push(Pilha, valor)
while (not isEmpty(Pilha))
   valor ← Pop(Pilha)
   write valor
```

Verifique este algoritmo para se convencer de que os valores estão, de fato, escritos em ordem reversa.

8.3 Filas

Uma fila é uma estrutura abstrata na qual os itens são inseridos em uma extremidade e removidos da outra extremidade. Os contadores chamam esse comportamento de FIFO – First-In, First-Out (que significa "primeiro a entrar, primeiro a sair"). Este TAD parece uma fila de espera em uma lotérica ou supermercado. De fato, as filas são usadas para simular esse tipo de situação. As inserções são feitas no final da fila e as remoções são feitas na frente da fila.

Outra maneira de descrever o comportamento de acesso de uma fila é dizer que o item removido é o item que está na fila há mais tempo. Visualizar uma fila dessa perspectiva é mais abstrato. Assim como a pilha, a operação de inserção não possui restrições; todo o comportamento FIFO é especificado na operação de remoção. Infelizmente, não existe uma terminologia padrão para filas relacionada às operações de inserção e exclusão. Enqueue, Enque, Enq, Enter e Insert são todos nomes usados para a operação de inserção. Dequeue, Deque, Deq, Delete e Remove são nomes usados para a operação de exclusão.

Aqui está um algoritmo que lê os valores dos dados e os imprime na ordem em que foram inseridos:

```
while (mais dados)
   read valor
   Enqueue(Fila, valor)
while (not isEmpty(Fila))
   valor ← Dequeue(Fila)
   write valor
```

? Polichinelos na sala de aula

O Instituto UTeach, um programa de formação de professores da Universidade do Texas, lançou uma iniciativa no final de 2013 para apoiar o uso de dispositivos móveis em sala de aula. Para demonstrar a tecnologia, alunos da Kealing Middle School, em Austin, fizeram polichinelos e, em seguida, colocaram os dedos na câmera de um tablet, que mediu suas frequências cardíacas. Os dados foram coletados diretamente em uma planilha projetada na parede. Os alunos puderam ver que, com o tempo, o exercício reduzia suas frequências cardíacas em repouso, de modo que seus corações bombeavam sangue com mais eficiência. A Verizon Wireless doou mais de US$ 1 milhão em equipamentos e auxílios financeiros para apoiar este trabalho no uso de tablets para o ensino.[1]

8.4 Listas

Listas ocorrem tão naturalmente na programação quanto na vida real. Manipulamos listas de convidados, listas de compras, listas de aulas e listas de coisas para fazer. A lista de listas é infinita. Três propriedades caracterizam as listas: os itens são homogêneos, os itens são lineares e as listas têm tamanhos variados. Por linear, queremos dizer que cada item, exceto o primeiro, tem um componente único que vem antes dele, e cada item, exceto o último, tem um componente único que vem depois dele. Por exemplo, se houver pelo menos três itens em uma lista, o segundo item vem depois do primeiro e antes do terceiro.

Enquanto pilhas e filas têm toda a semântica na operação de exclusão, listas geralmente fornecem operações para inserir um item (Insert), excluir um item (Delete), verificar se um item está lá (IsThere) e relatar o número de itens em uma lista (GetLength). Além disso, eles possuem algum mecanismo que permite ao usuário visualizar cada item em sequência (Reset, GetNext, MoreItems). Como os itens podem ser excluídos e recuperados, os itens na lista devem ser comparáveis.

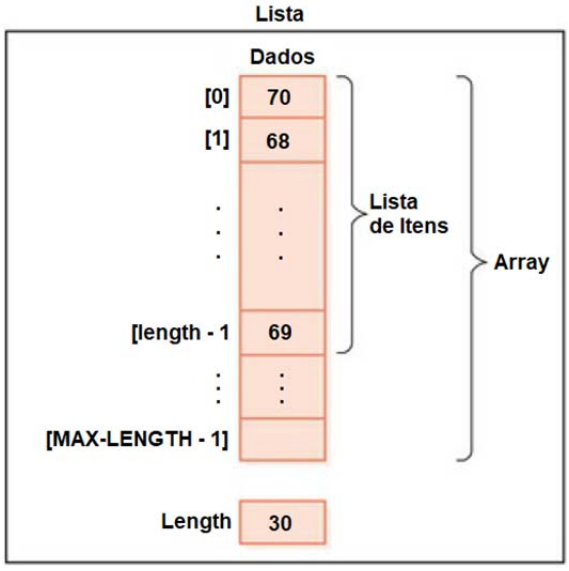


FIGURA 8.1 Uma lista desordenados de inteiros.

Não confunda uma lista com um array. Um array é uma estrutura interna; uma lista é uma estrutura abstrata. No entanto, uma lista pode ser implementada em um array, como mostrado na FIGURA 8.1. (ao lado)

Uma lista também pode ser visualizada como uma estrutura ligada (ou encadeada). Uma estrutura encadeada é baseada no conceito de nó. Um nó consiste em duas informações: os dados do usuário e um link ou ponteiro que indica onde encontrar o próximo nó. O final da lista é um link que contém nulo (NULL), indicado por uma barra '/'. Veja a FIGURA 8.2.

* Estrutura ligada (ou encadeada): uma implementação de um contêiner onde os itens são armazenados juntamente com informações sobre onde o próximo item pode ser encontrado.

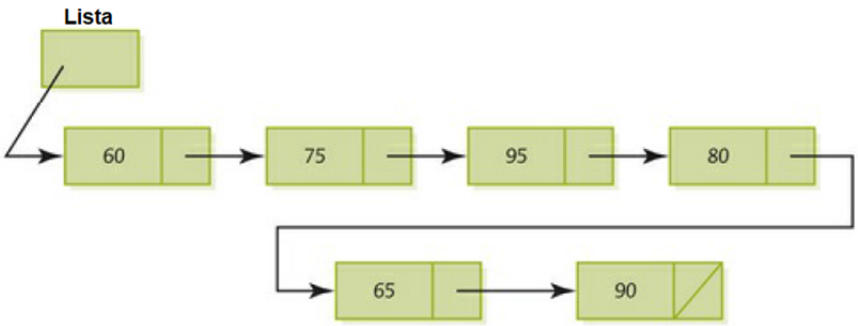


FIGURA 8.2 Uma lista encadeada não ordenada.

Listas não ordenadas são aquelas para as quais a ordem não é importante. Os itens são simplesmente colocados aleatoriamente nelas. Listas ordenadas são aquelas em que há uma relação semântica entre os itens na lista. Todos os itens, exceto o primeiro, vêm antes do próximo item neste tipo de lista sob algum relacionamento de ordenação. Todos os itens, exceto o último, vêm depois do anterior sob o mesmo relacionamento. As FIGURAS 8.3 e 8.4 visualizam as versões baseadas em matriz e encadeadas de uma lista ordenada.

Aqui está um algoritmo que lê valores de um arquivo e os coloca em uma lista. A lista é então impressa.

```
while (mais dados)
   read valor
   Insert(Lista, valor)
reset(Lista)
write "Os itens da lista são: "
while (MaisItens(Lista))
   getNext(Lista, nextItem)
   write nextItem
```

Usamos Reset, MoreItems e GetNext para iterar pela lista, retornando cada item em sequência. Se a lista for uma lista não ordenada, os itens serão impressos na ordem em que foram inseridos. Se a lista for ordenada, os itens serão impressos em ordem ordenada. Este algoritmo funciona independentemente da implementação da lista.

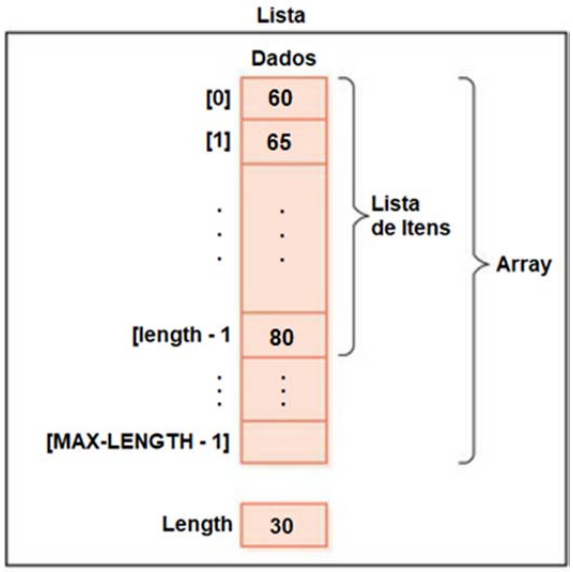


FIGURA 8.3 Uma lista ordenada de inteiros.

? Financiamento de hardware ponto a ponto

Hardware de código aberto é aquele que mantém seus designs disponíveis de forma semelhante ao código aberto em software. Dois defensores do hardware de código aberto, Justin Huynh e Matt Stack, fundaram o Open Source Hardware Bank (OSHB), usando dinheiro arrecadado de outros defensores do código aberto para apoiar projetos de hardware de código aberto. Os credores ponto a ponto recebem a promessa de 5% a 15% se o projeto for bem-sucedido.

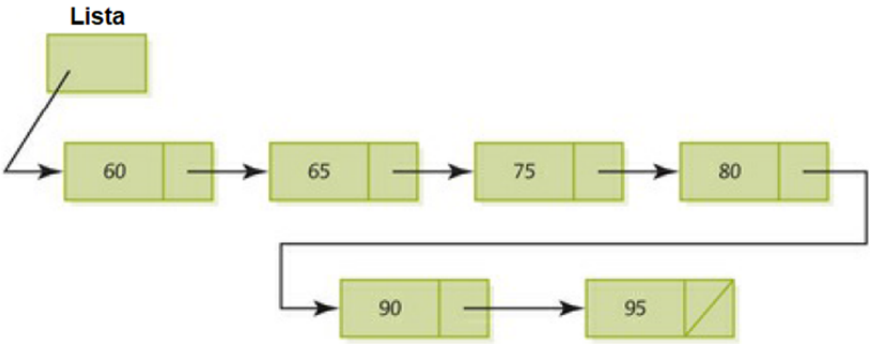


FIGURA 8.4 Uma lista encadeada ordenada.

8.5 Árvores

Estruturas abstratas como listas, pilhas e filas são lineares por natureza. Apenas um relacionamento nos dados está sendo modelado. Os itens estão próximos uns dos outros em uma lista ou estão próximos uns dos outros em termos de tempo em uma pilha ou fila. Descrever relacionamentos mais complexos requer estruturas mais complexas. Tomemos, por exemplo, relacionamentos familiares. Se quisermos modelar relacionamentos familiares em um programa, precisaríamos de uma estrutura hierárquica. Os pais apareceriam no topo da hierarquia, os filhos estariam no próximo nível, os netos no próximo nível e assim por diante (vide FIGURA 8.5).

Essas estruturas hierárquicas são chamadas de **árvores** (ou trees), e há uma rica teoria matemática relacionada a elas. Em computação, no entanto, frequentemente restringimos nossa discussão a árvores binárias. Em árvores binárias, cada nó da árvore não pode ter mais do que dois filhos.

Árvores Binárias

Uma árvore binária é uma estrutura abstrata na qual cada nó é capaz de ter dois nós sucessores, chamados **filhos**. Cada um dos filhos, sendo nós na árvore binária, também pode ter até dois nós filhos, e esses filhos também podem ter até dois filhos, e assim por diante, dando à árvore sua estrutura de ramificação. O início da árvore é um nó inicial único, chamado de raiz, que não é filho de nenhum nó. Veja a FIGURA 8.6.

Cada nó na árvore pode ter zero, um ou dois filhos. O nó à esquerda de um nó, se existir, é chamado de **filho esquerdo**. Por exemplo, na Figura 8.6, o filho esquerdo do nó raiz contém o valor 2. O nó à direita de um nó, se existir, é seu **filho direito**. O filho direito do nó raiz na Figura 8.6 contém o valor 3. Se um nó tiver apenas um filho, o filho pode estar em qualquer um dos lados, mas sempre estará em um lado específico. Na Figura 8.6, o nó raiz é o pai dos nós que contêm 2 e 3. (Livros didáticos anteriores usavam os termos filho esquerdo, filho direito e pai para descrever esses relacionamentos.) Se um nó na árvore não tem filhos, ele é chamado de **folha**. Na Figura 8.6, os nós que contêm 7, 8, 9 e 10 são nós folha.

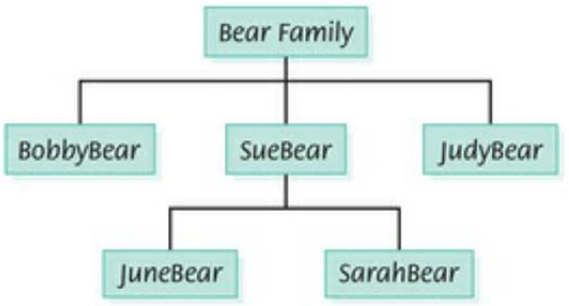


FIGURA 8.5 A árvore genealógica da família Bear.

* Árvore binária: Uma estrutura composta abstrata com um nó inicial único chamado raiz, na qual cada nó é capaz de ter dois nós filhos e na qual existe um caminho único da raiz para todos os outros nós.

* Raiz: O único nó inicial em uma árvore.

* Nó folha: Um nó da árvore que não possui filhos.

John von Neumann[2]



Cortesia do Departamento de Energia dos EUA

John von Neumann foi um brilhante matemático, físico, lógico e cientista da computação. Lendas foram transmitidas sobre sua memória impressionante e a velocidade fenomenal com que von Neumann resolvia problemas. Ele usou seus talentos não apenas para aprimorar suas teorias matemáticas, mas também para memorizar livros inteiros e recitá-los anos depois de tê-los lido. Mas pergunte a um policial rodoviário sobre a habilidade de von Neumann ao volante e ele provavelmente jogaria as mãos para o alto em desespero; ao volante, o gênio matemático era tão imprudente quanto um adolescente rebelde.

John von Neumann nasceu na Hungria em 1903, filho mais velho de um rico banqueiro judeu. Ele era capaz de dividir números de 8 dígitos de cabeça aos 6 anos de idade. Entrou para o ensino médio aos 11 anos, e não demorou muito para que seus professores de matemática recomendassem que ele tivesse aulas com professores universitários. Matriculou-se na Universidade de Berlim em 1921 para estudar química, em um acordo com seu pai, que queria que ele estudasse algo que lhe permitisse ganhar dinheiro. Em 1926, recebeu seu diploma em engenharia química pela Technische Hochschule de Zurique. No mesmo ano, doutorou-se em matemática pela Universidade de Budapeste, com uma tese sobre teoria dos conjuntos. De 1926 a 1929, von Neumann lecionou em Berlim e Hamburgo, enquanto era bolsista Rockefeller para estudos de pós-doutorado na Universidade de Göttingen.

Von Neumann chegou aos Estados Unidos no início da década de 1930 para lecionar em Princeton, mantendo seus cargos acadêmicos na Alemanha. Renunciou aos cargos alemães quando os nazistas chegaram ao poder; no entanto, não era um refugiado político como muitos eram naquela época. Enquanto estava em Princeton, trabalhou com o talentoso e ainda desconhecido estudante britânico Alan Turing. Continuou sua brilhante carreira matemática, tornando-se editor da revista Annals of Mathematics e coeditor da Compositio Mathematica. Durante a Segunda Guerra Mundial, von Neumann foi contratado como consultor para as Forças Armadas dos EUA e agências civis relacionadas devido ao seu conhecimento em hidrodinâmica. Também foi convocado para participar da construção da bomba atômica em 1943. Não foi surpresa que, após esse trabalho, o presidente Eisenhower o tenha nomeado para a Comissão de Energia Atômica em 1955.

Embora bombas e seu desempenho tenham fascinado von Neumann por muitos anos, um encontro fortuito em 1944 com Herbert Goldstine, um pioneiro que desenvolveu um dos primeiros computadores digitais eletrônicos operacionais, apresentou ao matemático algo mais importante do que bombas: computadores. A conversa casual de von Neumann com Goldstine em uma estação de trem despertou nele um novo fascínio. Ele começou a trabalhar no conceito de programa armazenado e concluiu que armazenar um programa internamente eliminava as horas de trabalho tedioso necessárias para reprogramar computadores (naquela época). Ele também desenvolveu uma nova arquitetura de computador para realizar essa tarefa de armazenamento. De fato, os computadores atuais são frequentemente chamados de

"máquinas de von Neumann" porque os princípios arquitetônicos que ele descreveu se mostraram tremendamente bem-sucedidos. As mudanças nos computadores nos últimos 40 anos ocorreram principalmente em termos de velocidade e composição dos circuitos fundamentais, mas a arquitetura básica projetada por von Neumann persistiu.

Durante a década de 1950, von Neumann foi consultor da IBM, onde revisou projetos de tecnologia avançada propostos e em andamento. Um desses projetos foi o FORTRAN de John Backus, que von Neumann teria questionado, perguntando por que alguém desejaria mais de uma linguagem de máquina. Em 1957, von Neumann morreu de câncer ósseo em Washington, D.C., aos 54 anos. Talvez seu trabalho com a bomba atômica tenha resultado no câncer ósseo que causou a morte de uma das mentes mais brilhantes e interessantes do século XX.

Além de especificar que um nó pode ter até dois filhos, a definição de uma árvore binária afirma que existe um caminho único da raiz para todos os outros nós. Em outras palavras, cada nó (exceto a raiz) tem um único pai.

Cada um dos filhos do nó raiz é, ele próprio, a raiz de uma árvore binária menor, ou subárvore. Na Figura 8.6, o filho esquerdo do nó raiz, contendo 2, é a raiz de sua subárvore esquerda, enquanto o filho direito, contendo 3, é a raiz de sua subárvore direita. De fato, qualquer nó na árvore pode ser considerado o nó raiz de uma subárvore. A subárvore cujo nó raiz tem o valor 2 também inclui os nós com valores 4 e 7. Esses nós são os descendentes do nó que contém 2. Os descendentes do nó que contém 3 são os nós com os valores 5, 6, 8, 9 e 10. Um nó é ancestral de outro nó se for o pai do nó ou o pai de algum outro ancestral daquele nó. (Sim, esta é uma definição recursiva.) Na Figura 8.6, os ancestrais do nó com o valor 9 são os nós que contêm 5, 3 e 1. Obviamente, a raiz da árvore é ancestral de todos os outros nós da árvore.

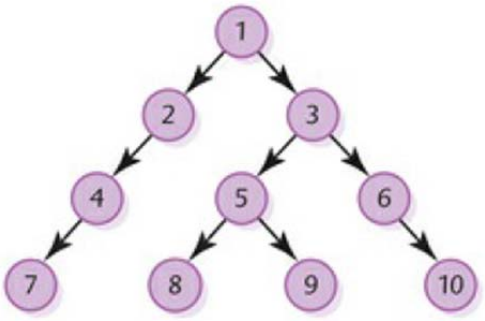


FIGURA 8.6 Uma árvore binária.

? Software de detecção de terrorismo

A análise de redes sociais fornece uma maneira de modelar como as pessoas interagem usando um ramo da matemática chamado teoria dos grafos. A teoria dos grafos mapeia as pessoas como nós e seus relacionamentos como elos. Atualmente, alguns pesquisadores estão utilizando essa abordagem para construir modelos de software de redes terroristas. Quando o software recebe dados sobre o número de membros de uma rede que foram presos, ele consegue estimar a probabilidade de a rede ter sido interrompida. Essa estimativa pode ser melhor do que aquela fornecida pelo julgamento humano.

Árvores Binárias de Busca

Uma árvore é análoga a uma lista não ordenada. Para encontrar um item na árvore, devemos examinar cada nó até encontrarmos o que queremos ou descobrirmos que ele não está na árvore. Uma **árvore binária de busca** é como uma lista ordenada, pois há uma ordenação semântica nos nós.

Uma árvore binária de busca tem a propriedade de forma de uma árvore binária; ou seja, um nó em uma árvore binária de busca pode ter zero, um ou dois filhos. Além disso, uma árvore binária de busca tem uma propriedade semântica que caracteriza os valores nos nós da árvore: o valor em qualquer nó é maior que o valor em qualquer nó em sua subárvore esquerda e menor que o valor em qualquer nó em sua subárvore direita. Veja a FIGURA 8.7.

Buscando em uma Árvore Binária de Busca

Vamos procurar o valor 18 na árvore mostrada na FIGURA 8.7. Comparamos 18 com 15, o valor no nó raiz. Como 18 é maior que 15, sabemos que, se 18 estiver na árvore, estará na subárvore direita da raiz. Observe a semelhança dessa abordagem com nossa busca binária de uma estrutura linear. Assim como na estrutura linear, eliminamos uma grande parte dos dados com uma comparação.

Em seguida, comparamos 18 com 17, o valor na raiz da subárvore direita. Como 18 é maior que 17, sabemos que se 18 estiver na árvore, estará na subárvore direita da raiz. Comparamos 18 com 19, o valor na raiz da subárvore direita. Como 18 é menor que 19, sabemos que se 18 estiver na árvore, estará na

subárvore esquerda da raiz. Comparamos 18 com 18, o valor na raiz da subárvore esquerda, e temos uma correspondência.

Agora, vejamos o que acontece quando buscamos um valor que não está na árvore. Vamos procurar 4 na Figura 8.7. Comparamos 4 com 15. Como 4 é menor que 15, se 4 estiver na árvore, estará na subárvore esquerda da raiz. Comparamos 4 com 7, o valor na raiz da subárvore esquerda. Como 4 é menor que 7, se 4 estiver na árvore, estará na subárvore esquerda de 7. Comparamos 4 com 5. Como 4 é menor que 5, se 4 estiver na árvore, estará na subárvore esquerda de 5. Comparamos 4 com 1. Como 4 é maior que 1, se 4 estiver na árvore, estará na subárvore direita de 1. Mas a subárvore direita de 1 está vazia, então sabemos que 4 não está na árvore.

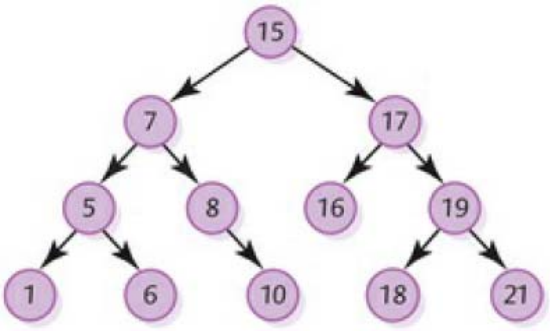


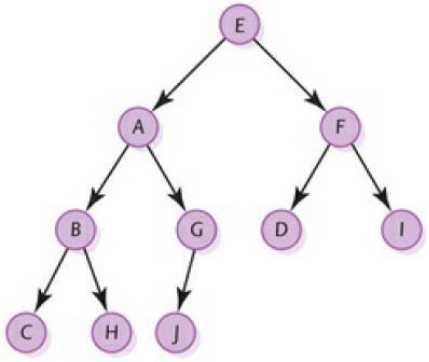
FIGURA 8.7 Uma árvore de busca binária.

Ao analisar os algoritmos que trabalham com árvores, usamos as seguintes convenções: se **currente** aponta para um nó, **info(currente)** refere-se aos dados do usuário no nó, **left(currente)** aponta para a raiz da subárvore esquerda de current e **right(current)** aponta para a raiz da subárvore direita de **currente**. O valor **NULL** é um valor especial que significa que o ponteiro não aponta para nada. Portanto, se um ponteiro contiver **NULL**, a subárvore estará vazia.

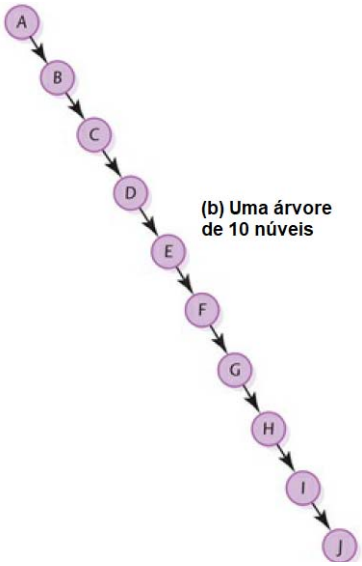
Usando essa notação, podemos agora escrever o algoritmo de busca. Começamos na raiz da árvore e avançamos para a raiz de subárvores sucessivas até encontrarmos o item que estamos procurando ou encontrarmos uma subárvore vazia. O item a ser buscado e a raiz da árvore (subárvore) são parâmetros – as informações de que o subalgoritmo precisa para ser executado.

```
isThere(tree, item)
   if (tree == NULL)
     return FALSE
   else if (item == info(tree))
     return TRUE
   else if (item < info(tree))
     isThere(left(tree), item)
   else
     isThere(right(tree), item)
```

A cada comparação, encontramos o item ou cortamos a árvore ao meio, movendo-a para buscar na subárvore esquerda ou na subárvore direita. Ao meio? Bem, não exatamente. O formato de uma árvore binária nem sempre é bem equilibrado. Claramente, a eficiência de uma operação de busca em uma árvore de busca binária está diretamente relacionada ao formato da árvore. Como a árvore adquire esse formato? O formato da árvore é determinado pela ordem em que os itens são inseridos na árvore. Observe a FIGURA 8.8. Na parte (a), a árvore de quatro níveis está comparativamente balanceada. Os nós poderiam ter sido inseridos em várias ordens diferentes para obter esta árvore. Em comparação, a árvore de dez níveis na parte (b) só poderia ter vindo dos valores inseridos em ordem.



(a) Uma árvore de 4 níveis.



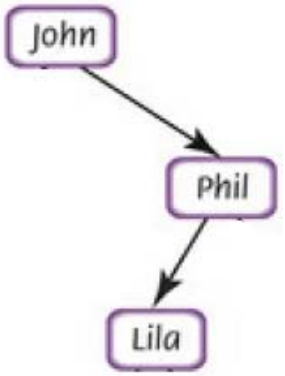
(b) Uma árvore de 10 núveis

FIGURA 8.8 Duas variações de uma árvore binária de busca.

Construindo uma Árvore Binária de Busca

Como construímos uma árvore binária de busca? Uma pista está no algoritmo de busca que acabamos de usar. Se seguirmos o caminho de busca e não encontrarmos o item, acabamos no local onde o item estaria se estivesse na árvore. Vamos agora construir uma árvore de busca binária usando as seguintes strings: **John**, **Phil**, **Lila**, **Kate**, **Becca**, **Judy**, **June**, **Mari**, **Jim**, **Sue**.

Como John é o primeiro valor a ser inserido, ele vai para a raiz. O segundo valor, Phil, é maior que John, então ele vai para a raiz da subárvore da direita. Lila é maior que John mas menor que Phil, então Lila vai para a raiz da subárvore da esquerda de Phil. A árvore se parece com a figura ao lado.



Kate é maior que John, mas menor que Phil e Lila, então Kate entra na raiz da subárvore esquerda de Lila. Becca é menor que John, então Becca entra na raiz da subárvore esquerda de John. Judy é maior que John, mas menor que Phil, Lila e Kate, então Judy entra na raiz da subárvore esquerda de Kate. Seguimos o mesmo caminho para June que fizemos para Judy. June é maior que Judy, então June entra na raiz da subárvore direita de Judy. Mari se torna a raiz da subárvore direita de Lila; Jim se torna a raiz da subárvore direita de Becca; e Sue se torna a raiz da subárvore direita de Phil. A árvore final é mostrada na FIGURA 8.9.

```
Insert(tree, item)
   if (tree == NULL)
      Colocar item na árvore
   else if (item < info(tree))
      Insert(left(tree), item)
   else
      Insert(right(tree), item)
```

A TABELA 8.1 mostra um rastreamento da inserção de **Nell** na árvore mostrada na FIGURA 8.9. Usamos o conteúdo da parte **info** do nó entre parênteses para indicar o ponteiro para a subárvore com esse valor como raiz.

Embora a ação "**Colocar item na árvore**" seja abstrata, não a expandimos. Precisaríamos saber mais sobre a implementação real da árvore para fazê-lo.

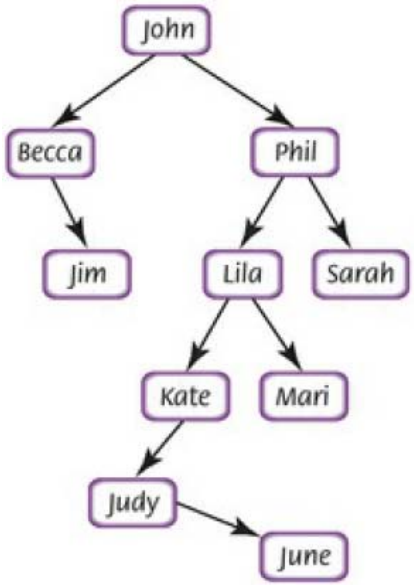


FIGURA 8.9 Uma árvore de busca binária construída a partir de strings.

Tabela 8.1 Rastreamento da inserção de Nell na árvore na Figura 8.9.

Chamada para insert	1º comando if	2º comando if	Ação ou chamada
`Insert((John), Nell)`	`(John) != NULL`	`Nell > John`	Insere na subárvore direita.
`Insert((Phil), Nell)`	`(Phil) != NULL`	`Nell > Phil`	Insere na subárvore esquerda.
`Insert((Lila), Nell)`	`(Lila) != NULL`	`Nell > Lila`	Insere na subárvore direita.
`Insert((Mari), Nell)`	`(Mari) != NULL`	`Nell > Mari`	Insere na subárvore direita.
`Insert((NULL), Nell)`	`(NULL) != NULL`		Armazena Nell como raiz da subárvore direita de Mari.

Imprimindo os Dados em uma Árvore Binária de Busca

Para imprimir o valor da raiz, precisamos primeiro imprimir todos os valores em sua subárvore esquerda, que por definição são menores que o valor na raiz. Depois de imprimir o valor na raiz, precisamos imprimir todos os valores na subárvore direita da raiz, que por definição são maiores que o valor na raiz. Então terminamos. Terminamos? Mas e os valores nas subárvores esquerda e direita? Como os imprimimos? Ora, da mesma forma, é claro. Afinal, eles são apenas árvores binárias de busca.

Este algoritmo parece fácil demais. Essa é a beleza dos algoritmos recursivos: eles são frequentemente curtos e elegantes (embora às vezes exijam algum esforço para serem rastreados). Vamos escrever e rastrear este algoritmo usando a árvore mostrada abaixo do algoritmo. Numeramos as chamadas em nosso rastreamento porque há duas chamadas recursivas. Veja a TABELA 8.2.

```
Print(tree)
   if (tree != NULL)
      Print(left(tree))      // Chamada recursiva R1
      write info(tree)
      Print(right(tree))     // Chamada recursiva R2
```

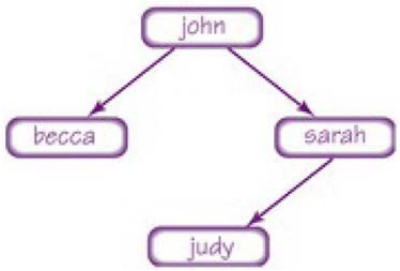


TABELA 8.2 Rastreamento da Impressão da Árvore Anterior.

Chamada	Qual chamada?	Comando if	Ação ou chamada
`Print((John))`	R1	`(John) != NULL`	Print(left(John))
`Print((Becca))`	R1	`(Becca) != NULL`	Print(left(Becca))
`Print(NULL)`	R1	`NULL != NULL`	Return
`Print((Becca))`			Print Becca, Print(right(Becca))
`Print(NULL)`	R2	`NULL != NULL`	Return, Return
`Print((John))`			Print John, Print(right(John))
`Print((Sarah))`	R2	`(Sarah) != NULL`	Print(left(Sarah))
`Print((Judy))`	R1	`(Judy) != NULL`	Print(left(Judy))
`Print(NULL)`	R1	`NULL != NULL`	Return
`Print((Judy))`			Print Judy, Print(right(Judy))
`Print(NULL)`	R2	`NULL != NULL`	Return, Return
`Print((Sarah))`			Print Sarah, Print(right(Sarah))
`Print(NULL)`	R2	`NULL != NULL`	Return, Return

Este algoritmo imprime os itens na árvore de busca binária em ordem crescente de valor. Outras travessias da árvore imprimem os itens em outras ordens. Nós os exploramos nos exercícios.

? Alarme falso

Uma falha em uma estação de rádio de Las Vegas alertou falsamente as operadoras de TV a cabo, rádio e TV em cinco condados sobre uma crise nacional inexistente. Esse erro ocorreu quando uma estação de rádio tentou enviar uma mensagem cancelando um Alerta AMBER anterior e, em vez disso, transmitiu um EAN, ou notificação de ação de emergência – um código especial reservado para o presidente dos Estados Unidos usar em caso de guerra nuclear ou emergência nacional extrema semelhante.

Outras Operações

A esta altura, você já deve ter percebido que uma árvore de busca binária é um objeto com a mesma funcionalidade de uma lista. A característica que separa uma árvore de busca binária de uma lista simples é a eficiência das operações; os outros comportamentos são os mesmos. Não mostramos o algoritmopara remover (o **Remove**), porque ele é muito complexo para este texto. Também ignoramos o conceito de comprimento (**length**) que deve acompanhar a árvore se ela for usada para implementar uma lista. Em vez de controlar o número de itens na árvore enquanto a construímos, vamos escrever um algoritmo que conta o número de nós na árvore.

Quantos nós há em uma árvore vazia? Zero. Quantos nós há em qualquer árvore? Há um mais o número de nós na subárvore esquerda e o número de nós na subárvore direita. Esta definição leva a uma definição recursiva da operação **Comprimento** (ou **Length**):

```
Length(tree)
   if (tree == NULL)
      return 0
   else
      return Length(left(tree)) + Length(right(tree)) + 1
```

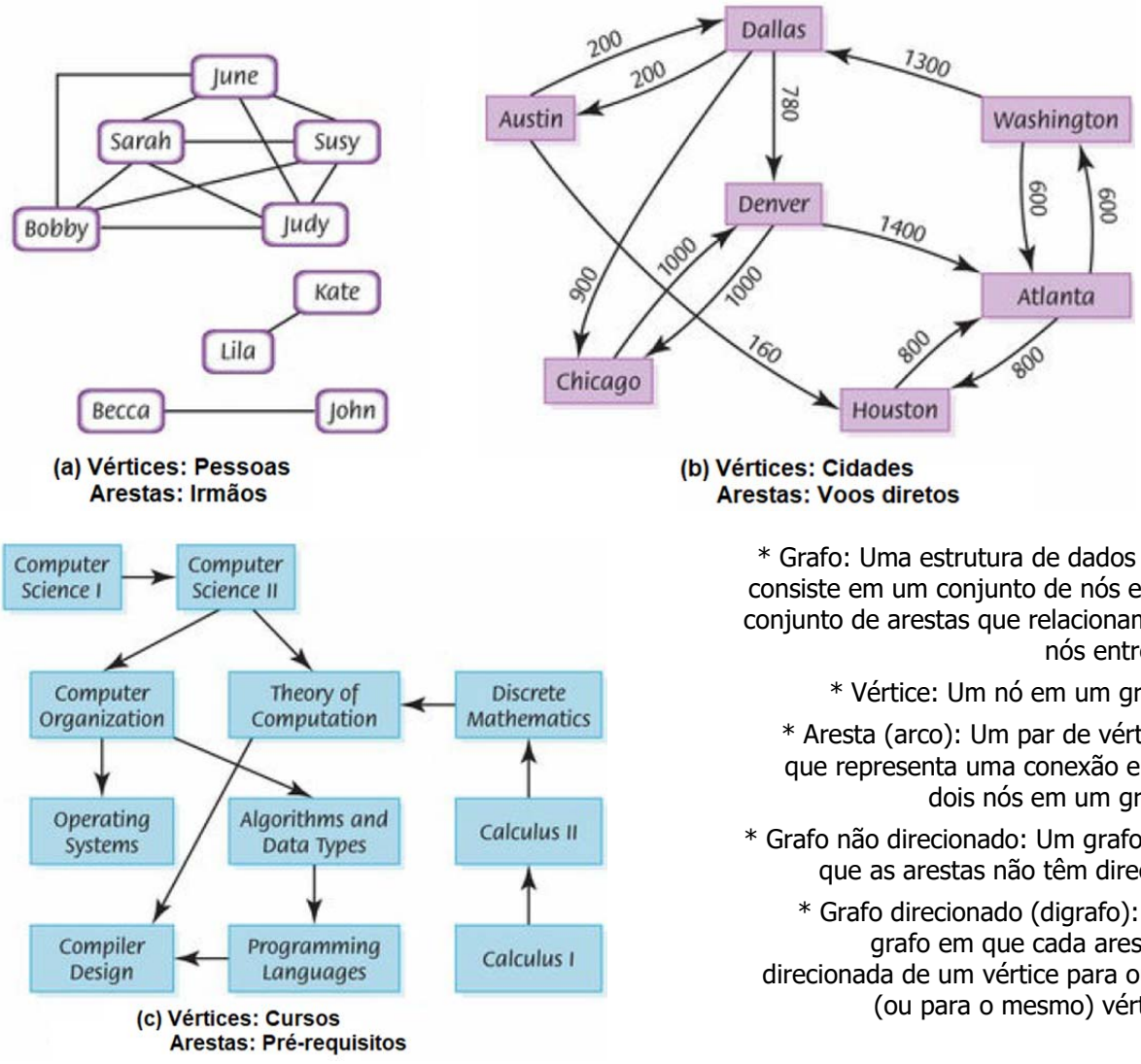
8.6 Grafos

A família Bear, como mostrado na FIGURA 8.5, representa apenas relacionamentos de pai e filho. Não há como determinar que BobbyBear, JuneBear e JudyBear são irmãos. Não seria bom poder representar outros tipos de relacionamentos, como irmão, primo, tia e assim por diante?

Árvores são uma maneira útil de representar relacionamentos nos quais existe uma hierarquia. Ou seja, um nó é apontado por no máximo um outro nó (seu pai). Se removermos a restrição de que cada nó pode ter apenas um nó pai, temos uma estrutura de dados chamada grafo. Um grafo é composto por um conjunto de nós chamados vértices e um conjunto de linhas chamadas arestas (ou arcos) que conectam os nós.

Os vértices no grafo representam objetos e as arestas descrevem relacionamentos entre esses objetos (os vértices). Por exemplo, se o grafo representa um mapa rodoviário, os vértices podem ser os nomes de cidades e as arestas que ligam os vértices podem representar estradas entre pares de cidades. Como as estradas que ligam as cidades são caminhos de mão dupla, as arestas neste grafo não têm direção. Tal grafo é chamado de grafo não direcionado. No entanto, se as arestas que ligam os vértices representam voos de uma cidade para outra, a direção de cada aresta é importante. A existência de um voo (aresta) de São Paulo para Salvador não garante a existência de um voo de Salvador para São Paulo. Um grafo cujas arestas são direcionadas de um vértice para outro é chamado de grafo direcionado (ou digrafo). Um grafo ponderado é aquele em que há valores anexados às arestas no grafo.

Observe os gráficos na FIGURA 8.10. As relações entre irmãos são não direcionadas. Por exemplo, June é irmã de Sarah e Sarah é irmã de June; veja a FIGURA 8.10(a). O gráfico de pré-requisitos na FIGURA 8.10(c) é direcionado: Computer Science I deve vir antes de Computer Science II. O cronograma de voo é direcionado e ponderado; veja a FIGURA 8.10(b). Há um voo de Dallas para Denver que cobre uma distância de 1.250 milhas, mas não há um voo direto de Denver para Dallas.



* Grafo: Uma estrutura de dados que consiste em um conjunto de nós e um conjunto de arestas que relacionam os nós entre si.

* Vértice: Um nó em um grafo.

* Aresta (arco): Um par de vértices que representa uma conexão entre dois nós em um grafo.

* Grafo não direcionado: Um grafo em que as arestas não têm direção.

* Grafo direcionado (digrafo): Um grafo em que cada aresta é direcionada de um vértice para outro (ou para o mesmo) vértice.

FIGURA 8.10 Exemplos de gráficos.

Se dois vértices são conectados por uma aresta, dizemos que são vértices adjacentes. Na FIGURA 8.10(a), June é adjacente a Bobby, Sarah, Judy e Susy. Um caminho de um vértice para outro consiste em uma sequência de vértices que os conectam. Por exemplo, há um caminho de Austin para Dallas, de Denver para Chicago. Não há um caminho de June até Lila, Kate, Becca ou John.

Os vértices representam quaisquer objetos que estejam sendo modelados: pessoas, casas, cidades, percursos, conceitos e assim por diante. As arestas representam relacionamentos entre esses objetos. Por exemplo, pessoas estão relacionadas a outras pessoas, casas estão na mesma rua, cidades estão conectadas por voos diretos, percursos têm pré-requisitos e conceitos são derivados de outros conceitos. (Veja a FIGURA 8.10.). Matematicamente, os vértices são o conceito indefinido sobre o qual a teoria dos grafos se baseia. Há muita matemática formal associada a grafos, que está além do escopo deste livro.

* Vértices adjacentes: Dois vértices conectados por uma aresta.

* Caminho: Uma sequência de vértices que conecta dois nós em um grafo.

Criando um Grafo

Listas, pilhas, filas e árvores são apenas contêineres. O usuário escolhe o que é mais apropriado para um problema específico. Não há semântica inerente além daquelas incorporadas ao processo de recuperação: uma pilha retorna o item que está na pilha há menos tempo; uma fila retorna o item que está na fila há mais tempo. Listas e árvores retornam as informações solicitadas. Um grafo, por outro lado, possui algoritmos definidos que, de fato, resolvem problemas clássicos. Primeiro, falamos sobre a construção de um grafo; depois, discutimos problemas que podem ser resolvidos usando um grafo.

Muitas informações são representadas em um grafo: os vértices, as arestas e os pesos. Vamos visualizar a estrutura como uma tabela usando os dados de conexão de voo. As linhas e colunas na TABELA 8.3 são rotuladas com os nomes das cidades. Um zero em uma célula indica que não há nenhum voo da cidade da linha para a cidade da coluna. Os valores na tabela representam o número de milhas da cidade da linha para a cidade da coluna.

TABELA 8.3 Dados para o Grafo de Voo.

	Atlanta	Austin	Chicago	Dallas	Denver	Houston	Washington
Atlanta	0	0	0	0	0	800	600
Austin	0	0	0	200	0	160	0
Chicago	0	0	0	0	1000	0	0
Dallas	0	200	900	0	780	0	0
Denver	1400	0	1000	0	0	0	0
Houston	800	0	0	0	0	0	0
Washington	600	0	0	1300	0	0	0

Para construir tal tabela, precisamos das seguintes operações:

- Adicionar um vértice à tabela.
- Adicionar uma aresta à tabela.
- Adicionar um peso à tabela.

Encontramos uma posição na tabela informando o nome da linha e o nome da coluna. Ou seja, (Atlanta, Houston) tem um voo de 800 milhas. (Houston, Austin) contém um zero, então não há nenhum voo direto de Houston para Austin.

Algoritmos de Grafos

Existem três algoritmos clássicos de busca definidos em um gráfico, cada um respondendo a uma pergunta diferente.

- Posso ir da Cidade X para a Cidade Y com minha companhia aérea favorita?
- Como posso voar da Cidade X para a Cidade Y com o menor número de escalas?
- Qual é o voo mais curto (em milhas) da Cidade X para a Cidade Y?

As respostas a essas três perguntas envolvem uma busca em profundidade, uma busca em largura e uma busca de caminho mais curto de origem única.

Busca em Profundidade (Depth-First Search - DFS)

Posso ir da Cidade X para a Cidade Y com minha companhia aérea favorita? Dado um vértice inicial e um vértice final, vamos desenvolver um algoritmo que encontre um caminho do **verticeInicial** ao **verticeFinal**. Precisamos de uma maneira sistemática de rastrear as cidades enquanto as investigamos. Vamos usar uma pilha para armazenar vértices à medida que os encontramos na tentativa de encontrar um caminho entre os dois vértices. Com uma busca em profundidade, examinamos o primeiro vértice adjacente ao **verticeInicial**; se este for o **verticeFinal**, a busca termina. Caso contrário, examinamos todos os vértices que podem ser alcançados em uma única etapa a partir deste primeiro vértice.

Enquanto isso, precisamos armazenar os outros vértices adjacentes ao **verticeInicial** para usar posteriormente, se precisarmos deles. Se não existir um caminho a partir do primeiro vértice adjacente ao **verticeInicial**, voltamos e tentamos o segundo vértice, o terceiro vértice e assim por diante. Como queremos viajar o máximo possível por um caminho, voltando se o **verticeFinal** não for encontrado, uma pilha é a estrutura apropriada para armazenar os vértices.

? Quem precisa de bancos?

Um clube de empréstimo é uma comunidade financeira online que reúne tomadores de empréstimo e investidores. Clubes de investimento existem há muito tempo, mas agora a internet permite que pessoas que nunca se conheceram formem clubes. Clubes de empréstimo investem em imóveis, empresas de software e pequenas empresas. O Lending Club, o maior clube desse tipo nos Estados Unidos, está prestes a realizar uma oferta pública inicial em 2014. Juntamente com o PayPal, os clubes de empréstimo estão tentando reduzir os custos de empréstimo para empresas da Main Street que não se qualificam para empréstimos bancários.[3]

Marcamos um vértice como visitado depois de colocar todos os seus vértices adjacentes na pilha. Se processarmos um vértice que já foi visitado, continuamos colocando os mesmos vértices na pilha repetidamente. Então, o algoritmo não é um algoritmo, porque pode nunca terminar. Portanto, não devemos processar um vértice mais de uma vez.

Vamos aplicar esse algoritmo ao grafo de rota aérea de exemplo na FIGURA 8.10(b). Queremos voar de Austin para Washington. Inicializamos nossa busca colocando nossa cidade inicial na pilha (FIGURA 8.11(a)). No início do loop, retiramos a cidade atual, Austin, da pilha. Os lugares que podemos alcançar diretamente de Austin são Dallas e Houston; colocamos ambos os vértices na pilha (FIGURA 8.11(b)). No início da segunda iteração, retiramos o vértice superior da pilha – Houston. Houston não é nosso destino, então retomamos nossa busca a partir daí. Há apenas um voo saindo de Houston, para Atlanta; colocamos Atlanta na pilha (FIGURA 8.11(c)). Novamente, removemos o vértice superior da pilha. Atlanta não é nosso destino, então continuamos a busca a partir daí. Atlanta tem voos para duas cidades: Houston e Washington.

```
DFS(verticeInicial, verticeFinal)
   achou ← FALSE
   Push(Pilha, verticeInicial)
   while (not estaVazia(Pilha) and not achou)
      verticeTemp ← Pop(Pilha)
      if (verticeTemp == verticeFinal)
         write verticeFinal
         achou ← TRUE
      else if (verticeTemp não foi visitado)
         write verticeTemp
         Empilhe todos os vértices adjacentes não visitados com verticeTemp
         Marque verticeTemp como visitado
   if (achou)
      write "O caminho foi impresso"
   else
      write "O caminho não existe"
```

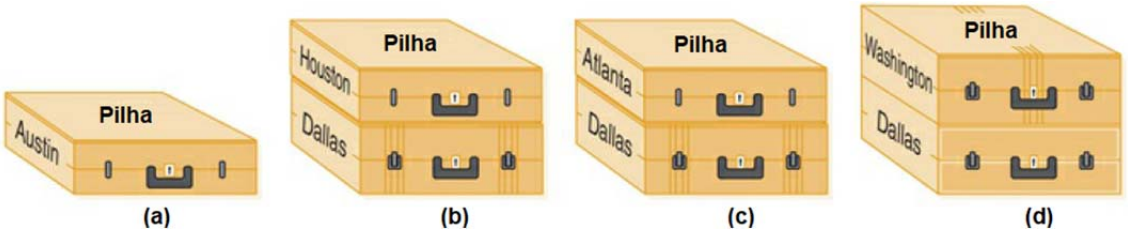


FIGURA 8.11 Usando uma pilha para armazenar as rotas

Mas acabamos de chegar de Houston! Não queremos voar de volta para cidades que já visitamos; isso poderia causar um loop infinito. Mas já resolvemos este problema: Houston já foi visitada, então continuamos sem colocar nada na pilha. O segundo vértice adjacente, Washington, não foi visitado, então o colocamos na pilha (FIGURA 8.11(d)). Novamente, removemos o vértice superior da pilha. Washington é nosso destino, então a busca está completa.

A FIGURA 8.12 (ao lado) mostra o resultado da pergunta se podemos chegar a Washington a partir de Austin.

Essa busca é chamada de busca em profundidade porque vamos até o ramo mais profundo, examinando todos os caminhos que começam em Houston antes de retornarmos para a busca a partir de Dallas. Quando precisar voltar, você pega o ramo mais próximo de onde chegou ao fim. Ou seja, você vai o mais longe possível por um caminho antes de tomar alternativas nos ramos anteriores.

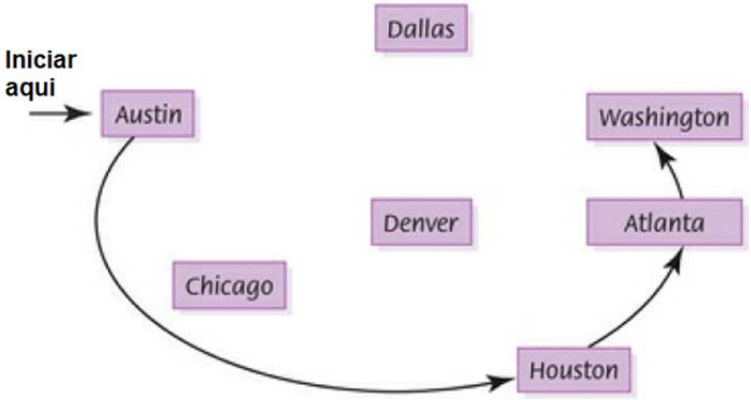


FIGURA 8.12 A busca em profundidade.

Busca em Largura (Breadth-First Search - BFS)

Como ir da Cidade X para a Cidade Y com o menor número de paradas? A travessia em largura responde a essa pergunta. Quando chegamos a um beco sem saída em uma busca em profundidade, retrocedemos o mínimo possível. Tentamos outra rota a partir do vértice mais recente — a rota no topo da nossa pilha. Em uma busca em largura, queremos retroceder o máximo possível para encontrar uma rota originada nos vértices mais antigos. A pilha não é a estrutura correta para encontrar uma rota mais antiga. Ela rastreia as coisas na ordem oposta de sua ocorrência — ou seja, a rota mais recente está no topo. Para rastrear as coisas na ordem em que acontecem, usamos uma fila. A rota na frente da fila é uma rota de um vértice anterior; a rota no final da fila é de um vértice posterior. Assim, se substituirmos uma pilha por uma fila, obteremos a resposta para nossa pergunta.

```
BFS(verticeInicial, verticeFinal)
   achou ← FALSE
   Enqueue(Fila, verticeInicial)
   while (not estaVazia(Fila) and not achou)
      verticeTemp ← Dequeue(Fila)
      if (verticeTemp == verticeFinal)
         write verticeFinal
         achou ← TRUE
      else if (verticeTemp não foi visitado)
         write verticeTemp
         Enfileire os vértices adjacentes não visitados com verticeTemp
         Marque verticeTemp como visitado
   if (achou)
      write "O caminho foi impresso"
   else
      write "O caminho não existe"
```

Vamos aplicar este algoritmo ao mesmo gráfico de rotas aéreas da Figura 8.10(b). Qual caminho nos fornece a rota de Austin para Washington com o menor número de paradas? Austin está na fila para iniciar o processo (FIGURA 8.13(a)). Retiramos Austin da fila e enfileiramos todas as cidades que podem ser acessadas diretamente de Austin: Dallas e Houston (FIGURA 8.13(b)). Em seguida, retiramos da fila o seu elemento da frente. Dallas não é o destino que buscamos, então enfileiramos todas as cidades adjacentes que ainda não foram visitadas: Chicago e Denver (FIGURA 8.13(c)). (Austin já foi visitada, portanto, não está na fila.) Novamente, retiramos o elemento da frente da fila. Este elemento é a outra cidade com "uma parada" – Houston. Houston não é o destino desejado, então continuamos a busca. Há apenas um voo saindo de Houston, e é para Atlanta. Como nunca visitamos Atlanta antes, ela está na fila (FIGURA 8.13(d)).

Agora sabemos que não podemos chegar a Washington com uma parada, então começamos a examinar as conexões com duas paradas. Retiramos Chicago da fila; este não é o nosso destino, então colocamos sua cidade adjacente, Denver, na fila (FIGURA 8.13(e)). Agora, esta é uma situação interessante: Denver está na fila duas vezes. Colocamos Denver na fila em uma etapa e removemos sua entrada anterior na etapa

seguinte. Denver não é nosso destino, então colocamos suas cidades adjacentes que não foram visitadas (apenas Atlanta) na fila (FIGURA 8.13(f)). Esse processamento continua até que Washington seja colocada na fila (de Atlanta) e seja finalmente retirada da fila. Encontramos a cidade desejada e a busca está concluída (FIGURA 8.14).

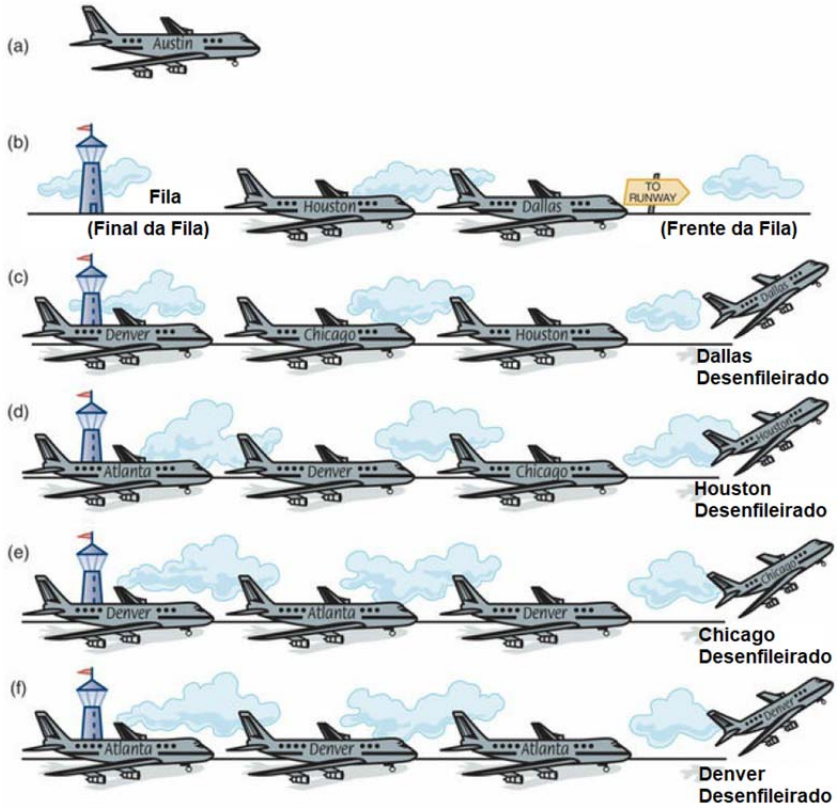


FIGURA 8.13 Usando uma fila para armazenar as rotas.

Como você pode ver nesses dois algoritmos, uma busca em profundidade percorre o caminho mais longe possível a partir do verticeInicial antes de procurar um caminho que comece no segundo vértice adjacente ao verticeInicial. Em contraste, uma busca em largura examina todos os vértices adjacentes a verticeInicial antes de procurar aqueles adjacentes a esses vértices.

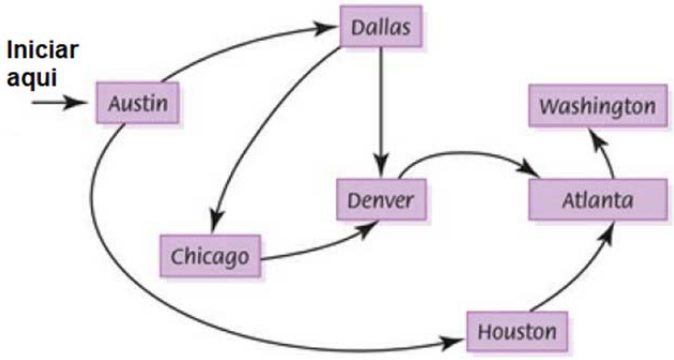
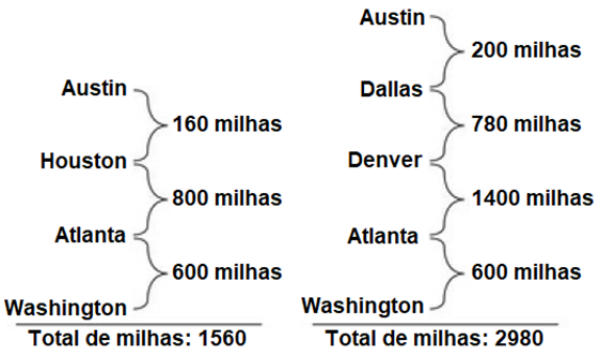


FIGURA 8.14 Busca em largura.

Busca de Caminho Mais Curto de Origem Única

Qual é o voo mais curto (em milhas) de Austin para alguma outra cidade? Sabemos, pelas duas operações de busca que acabamos de discutir, que pode haver vários caminhos de um vértice para outro. Suponha que queremos encontrar o caminho mais curto (shortest path) de Austin para cada uma das outras cidades que sua companhia aérea favorita atende. Por "caminho mais curto", queremos dizer o caminho cujos valores de aresta (pesos), quando somados, têm a menor soma. Considere os dois caminhos ao lado de Austin para Washington.



É claro que o primeiro caminho é preferível, a menos que você queira acumular milhas de passageiro frequente.

Vamos desenvolver um algoritmo que exiba o caminho mais curto de uma cidade inicial designada para **todas as outras cidades** no grafo – desta vez, não estamos procurando um caminho entre uma cidade inicial e uma cidade final. Como nas duas buscas descritas no grafo anterior, precisamos de uma estrutura auxiliar para armazenar cidades que processaremos posteriormente. Ao recuperar a cidade que foi colocada mais recentemente na estrutura, a busca em profundidade tenta continuar "avançando". Ela tenta uma solução de um voo, depois uma solução de dois voos, depois uma solução de três voos e assim por diante. Ela retorna para uma solução com menos voos somente quando chega a um beco sem saída. Ao recuperar a cidade que estava na estrutura há mais tempo, a busca em largura tenta todas as soluções de um voo, depois todas as soluções de dois voos e assim por diante. A busca em largura encontra um caminho com um número mínimo de voos.

É claro que o **número mínimo** de voos não significa necessariamente a **distância total** mínima. Ao contrário das buscas em profundidade e em largura, o percurso pelo caminho mais curto deve levar em conta o número de milhas (pesos de aresta) entre as cidades em sua busca. Queremos recuperar o vértice mais próximo do vértice atual – ou seja, o vértice conectado com o peso de aresta mínimo. No contêiner abstrato chamado **fila de prioridades**, o item recuperado é o item na fila com a prioridade mais alta. Se priorizarmos milhas, podemos enfileirar itens compostos por um registro que contém dois vértices e a distância entre eles.

Este algoritmo é muito mais complexo do que vimos antes, então paramos por aqui. No entanto, o leitor matematicamente aventureiro pode continuar buscando esta solução.

8.7 Subprogramas

Quando examinamos a recursão, introduzimos o conceito de um subalgoritmo nomeado. Aqui, o examinamos no contexto não recursivo e discutimos como passamos informações de um lado para o outro entre algoritmo e subalgoritmo. Como estamos falando de construções reais de linguagem de programação, chamamos essas estruturas de **subprogramas** em vez de **subalgoritmos**.

Muitos subprogramas estão disponíveis como parte de uma linguagem de alto nível ou como parte da biblioteca que acompanha a linguagem. Por exemplo, problemas matemáticos frequentemente precisam calcular funções trigonométricas. Subprogramas que calculam esses valores estão disponíveis na maioria das linguagens de alto nível, de uma forma ou de outra. Quando um programa precisa calcular um desses valores, o programador procura o nome do subprograma que calcula o valor e apenas chama o subprograma para realizar o cálculo.

Se um desses subprogramas precisar receber informações, a unidade chamadora envia os valores para o subprograma usar. Por exemplo, as duas instruções a seguir definem **x** como **m** vezes a função seno de **t** e **y** como o valor absoluto de **z**. A função seno e a função valor absoluto são incorporadas em muitas linguagens. A informação enviada para a função seno é **t**; a informação enviada para a função valor absoluto é **z**. Ambas as funções são subprogramas que retornam valores.

```
x ← m * sin(t)
y ← abs(z)
```

O mesmo se aplica quando você escreve seus próprios subprogramas. Agora, examinamos o mecanismo usado para passar informações entre o programa chamador e o subprograma.

Assumimos essas capacidades nos algoritmos relacionados aos tipos de dados abstratos que examinamos. Tomemos, por exemplo, o seguinte algoritmo de lista.

```
while (mais dados)
   read valor
   Insert(Lista, valor)
Reset(Lista)
write "Os itens da lista são: "
while (maisItens(Lista))
   getNext(Lista, proximoItem)
   write proximoItem
```

A função **Insert** precisa de uma lista e de um valor para adicionar a ela. A função **Reset** precisa da lista para ser reinicializada. A função **maisItens** precisa da lista para verificar se ainda há mais itens a serem retornados. A função **getNext** precisa da lista como entrada e retorna o próximo item da lista. Essa comunicação é feita por meio do conceito de **lista de parâmetros**.

Passagem de Parâmetros

Uma lista de parâmetros é uma lista de identificadores ou valores com os quais o subprograma deve trabalhar; ela aparece entre parênteses ao lado do nome do subprograma. Como um subprograma é definido antes de ser chamado, ele não sabe com quais variáveis da unidade chamadora deve trabalhar. Para resolver esse dilema, especificamos uma lista de nomes de variáveis entre parênteses ao lado do nome do subprograma. Esses identificadores são chamados de parâmetros. Quando o subprograma é chamado, a unidade chamadora lista o nome do subprograma, seguido por uma lista de identificadores entre parênteses. Esses identificadores são chamados de argumentos. Os argumentos representam variáveis reais na unidade chamadora com a qual o subprograma deve trabalhar.

* Lista de Parâmetros: Um mecanismo de comunicação entre duas partes de um programa.

* Parâmetros: Os identificadores listados entre parênteses ao lado do nome do subprograma; às vezes chamados de parâmetros formais.

* Argumentos: Os identificadores listados entre parênteses na chamada do subprograma; às vezes chamados de parâmetros reais.

Você pode pensar em um parâmetro como um identificador temporário usado dentro de um subprograma. Quando um subprograma é chamado, a unidade que o chama envia os nomes dos identificadores reais que o subprograma deve usar. A ação no subprograma é definida usando os parâmetros; a ação é executada usando os argumentos. Quando a ação ocorre, os argumentos são substituídos um a um pelos parâmetros. Essa substituição pode ser feita de várias maneiras, mas a prática mais comum é por posição. O primeiro argumento substitui o primeiro parâmetro, o segundo argumento substitui o segundo parâmetro e assim por diante.

Prometemos não analisar muitas implementações, mas esta é fácil. Podemos implementar uma lista usando um array e um campo de comprimento. Quando adicionamos um item à lista, o armazenamos no array (**valores**) na posição **length – 1** e incrementamos **length**. Vinculamos os **valores** e o comprimento (**length**) em um registro chamado **lista**, que passamos para o subprograma que precisa dele.

```
Insert(Lista, item)                          // Definição do subprograma
   Lista.valores[Lista.length - 1] ← item
   Lista.length ← Lista.length + 1

Insert(NovaLista, valor)                     // Comando chamador
```

A variável **Lista** é o parâmetro e a variável **NovaLista** é o argumento. Quando o comando **Insert** é executado, **NovaLista** substitui **Lista**.

O mecanismo de substituição funciona de forma muito semelhante a um quadro de mensagens. Quando um subprograma é chamado, uma lista de argumentos é fornecida ao subprograma (colocada no quadro de mensagens do subprograma). Esses argumentos informam ao subprograma onde encontrar os valores a serem usados. Quando um parâmetro é usado no corpo do subprograma, o subprograma acessa o argumento por meio de sua posição relativa no quadro de mensagens. Ou seja, o subprograma procura seu primeiro parâmetro na primeira posição no quadro de mensagens e seu segundo parâmetro na segunda posição no quadro de mensagens. Veja a FIGURA 8.15.

O número de argumentos na chamada deve corresponder ao número de parâmetros no cabeçalho do subprograma. Como os argumentos e parâmetros são correspondidos por posição, seus nomes não precisam ser os mesmos. Isso é muito útil quando um subprograma é chamado mais de uma vez, com argumentos diferentes em cada chamada. Parâmetros passados dessa forma são frequentemente chamados de **parâmetros posicionais**.



FIGURA 8.15 Passando parâmetros

? Hackers e crackers

A palavra hacker costumava ser um elogio, descrevendo um programador capaz de escrever programas muito sofisticados quase da noite para o dia. Depois, o termo passou a se referir a alguém que mexia em programas com más intenções. Agora, cracker se refere à pessoa com más intenções, e hacker retornou à sua definição original.

Parâmetros por Valor e por Referência

Existem duas maneiras básicas de passar parâmetros: por valor e por referência (ou endereço). Se um parâmetro for um parâmetro de valor, a unidade que faz a chamada fornece uma **cópia** do argumento para o subprograma. Se um parâmetro for um parâmetro de referência, a unidade que faz a chamada fornece o **endereço** de memória do argumento para o subprograma. Essa diferença fundamental significa que um subprograma não pode alterar o conteúdo de um argumento de valor porque recebe apenas uma cópia do argumento. O subprograma pode modificar a cópia, mas a variável original não será alterada. Em contraste, qualquer argumento passado pela unidade que faz a chamada para um parâmetro de referência pode ser alterado pelo subprograma porque o subprograma manipula a variável real, não uma cópia dela. No exemplo anterior, o registro passado como **Lista** deve ser um parâmetro de referência. Se não for, os itens serão inseridos na cópia, não no original.

* Parâmetro de valor: Um parâmetro que espera que uma cópia de seu argumento seja passada pela unidade que faz a chamada (colocada no quadro de mensagens).

* Parâmetro de referência: Um parâmetro que espera que o endereço do seu argumento seja passado pela unidade de chamada (colocado no quadro de mensagens).

Pense na diferença desta forma: para acessar um parâmetro de referência, o subprograma acessa o conteúdo do **endereço** listado no quadro de mensagens. Para acessar um parâmetro de valor, o subprograma acessa o **conteúdo** do quadro de mensagens. Claramente, tanto a unidade chamadora quanto o subprograma devem saber qual parâmetro/argumento deve ser passado por valor e qual deve ser passado por referência. Nem todas as linguagens de alto nível permitem ambos os tipos de parâmetros, mas aquelas que permitem possuem alguns esquemas sintáticos para rotular parâmetros como valor ou referência.

Antes de deixarmos os subprogramas, vejamos um exemplo que ilustra a diferença entre parâmetros de valor e de referência. Já escrevemos um algoritmo que troca o conteúdo de dois lugares na memória. Aqui está a solução sem nomes de variáveis dependentes do problema.

```
Troca(item1, item2)
   temp ← item2
   item2 ← item1
   item1 ← temp
```

Agora, suponha que a unidade chamadora (a parte do programa que deseja que o conteúdo dos dois lugares seja trocado) chame **Troca** com **dado** e **dado2** como parâmetros.

```
Troca (dado1, dado2)        // Comando de chamada
```

Agora, digamos que **dado1** esteja armazenado no local 0002 e **dado2** esteja armazenado no local 0003. Esses locais contêm os valores 30 e 40, respectivamente. A FIGURA 8.16 mostra o conteúdo do quadro de mensagens quando os parâmetros são passados por valor e por referência. Quando um parâmetro é um parâmetro de valor, o subprograma sabe como manipular o valor no quadro de mensagens. Quando um parâmetro é um parâmetro de referência, o subprograma sabe como manipular o conteúdo do endereço no quadro de mensagens. Os parâmetros do subprograma **Troca** devem ser parâmetros de valor ou de referência?

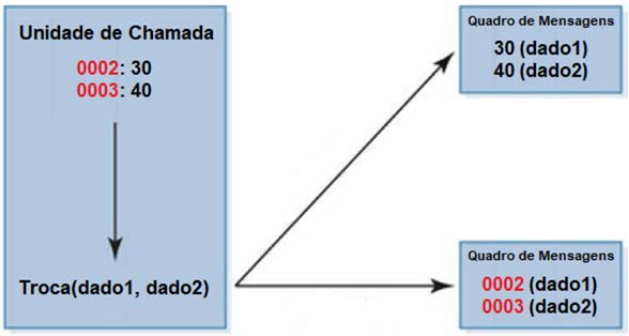


FIGURA 8.16 A diferença entre parâmetros de valor e parâmetros de referência.

Antes de deixarmos o tópico de subprogramas e parâmetros, vamos implementar mais três subprogramas de lista: **getLength**, **isThere** e **Delete**. Se os itens da lista não devem ser mantidos em ordem de classificação, podemos simplesmente colocar o primeiro na posição **length** e incrementar **length**. Para este exemplo, vamos supor que apenas uma cópia de item possa estar na lista.

```
getLength(Lista)
   return Lista.length
```

```
isThere(Lista, item)
   posição ← 0
   while (posição < Lista.length and Lista.valores[posição] != item)
      posição ← posição + 1
   return posição < Lista.length
```

A função **isThere** é um subprograma que retorna um valor – neste caso, um valor booleano. Portanto, na chamada, seria usado em uma expressão como

```
if (isThere(Lista, item))
   write item + "está na lista"
```

Este tipo de subprograma é chamado de subprograma que retorna valor. Os subprogramas **Delete** e **Insert**, por outro lado, não retornam um valor específico. No entanto, eles retornam a lista alterada por meio de seus parâmetros. Se assumirmos que o item a ser excluído está na lista, a implementação é simples: quando encontramos o item a ser excluído, simplesmente o trocamos pelo último item da lista e decrementamos seu comprimento.

```
Delete(Lista, item)
   posição ← 1
   while (Lista.valores[posição] != item)
      posição ← posição + 1
   Troca (Lista.valores[Lista.length - 1], Lista.valores[posição])
   return Lista.length ← Lista.length – 1
```

O subprograma **IsThere** pode ser usado para garantir que o item a ser excluído esteja na lista.

```
if (isThere(Lista, item))
   Delete (Lista, item)
```

Subprogramas que retornam valor incluem a instrução **return** seguida por um valor a ser retornado. Os subprogramas que não retornam valor podem ter uma instrução **return**, mas ela é desnecessária. Para concluir esta seção, aqui está um segmento de código que lê valores na lista e, em seguida, exclui alguns valores.

```
while (mais valores)
   read aValor
   if (not isThere(Lista, aValor))
      Insert(Lista, aValor)
write "Valor a excluir (ou digite Sair): "
read aValor
if (aValor != "Sair")
   if (isThere(Lista, aValor))
      Delete(Lista, aValor)
```

RESUMO

Listas, pilhas, filas, árvores e grafos são estruturas abstratas compostas úteis. Cada uma tem sua própria propriedade definidora e as operações que garantem essa propriedade. Todas essas estruturas abstratas incluem operações para inserir e remover itens. As listas e árvores também possuem operações para encontrar itens dentro da estrutura.

As listas e árvores têm as mesmas propriedades: itens podem ser inseridos, excluídos e recuperados. Itens podem ser inseridos em uma pilha, mas o item removido e retornado é o último item inserido na pilha – ou seja, o item que está na pilha há menos tempo. Itens podem ser inseridos em uma fila, mas o item removido e retornado é o primeiro item colocado na fila – ou seja, o item que está na fila há mais tempo.

As listas, pilha, filas e árvores são meramente estruturas de contenção, mas grafos são mais complexos. Uma grande variedade de algoritmos matemáticos pode ser aplicada às informações em um grafo. Examinamos três deles: a busca em largura, a busca em profundidade e a busca de caminho mais curto de origem única.

Instruções de subprograma permitem que subalgoritmos sejam implementados independentemente. Um subprograma pode retornar valor, caso em que é chamado colocando seu nome e argumentos dentro de uma expressão. Alternativamente, um subprograma pode não retornar valor (void), caso em que o nome do subprograma é usado como uma instrução no programa chamador. Os dados enviados de/para subprogramas são transmitidos pelo uso de listas de parâmetros. Os parâmetros podem ser parâmetros de referência ou de valor. Um argumento é passado para um parâmetro de valor enviando uma cópia do argumento para o subprograma. Um argumento é passado para um parâmetro de referência enviando o endereço do argumento para o subprograma.

EXERCÍCIOS

Para os Exercícios 1 a 10, indique qual estrutura seria a escolha mais adequada para cada uma das seguintes aplicações, marcando-as da seguinte forma:

A. Pilha B. Fila C. Árvore D. Árvore de busca binária E. Gráfico

1. Uma simulação bancária da operação do caixa para verificar como os tempos de espera seriam afetados pela adição de outro caixa.
2. Um programa para receber dados que devem ser salvos e processados na ordem inversa.
3. Uma agenda eletrônica, mantida ordenada por nome.
4. Um processador de texto com uma tecla PF que faz com que o comando anterior seja reexibido; toda vez que a tecla PF for pressionada, o programa deve mostrar o comando que precedeu o que estava sendo exibido.
5. Um dicionário de palavras usado por um corretor ortográfico a ser criado e mantido.
6. Um programa para acompanhar os pacientes enquanto eles se registram em uma clínica médica, atribuindo os aos médicos por ordem de chegada.
7. Um programa que monitora a localização de alimentos enlatados em uma prateleira.
8. Um programa para monitorar os times de futebol em um torneio municipal.
9. Um programa para monitorar os relacionamentos familiares.
10. Um programa para manter as rotas de uma companhia aérea.

Para os Exercícios 11 a 30, marque as respostas como verdadeiras ou falsas, da seguinte forma:

11. Uma busca binária não pode ser aplicada a uma árvore.
12. Uma pilha e uma fila são nomes diferentes para o mesmo TAD.
13. Uma pilha exibe comportamento FIFO.
14. Uma fila exibe comportamento LIFO.
15. Uma folha em uma árvore é um nó sem filhos.
16. Uma árvore binária é uma árvore na qual cada nó pode ter zero, um ou dois filhos.
17. Árvore binária de busca é outro nome para uma árvore binária.
18. O valor no filho direito de um nó (se existir) em uma árvore binária de busca será maior que o valor no próprio nó.
19. O valor no filho esquerdo de um nó (se existir) em uma árvore binária de busca será maior que o valor no próprio nó.
20. Em um grafo, os vértices representam os itens que estão sendo modelados.
21. Algoritmos que usam uma lista devem saber se a lista é baseada em matrizes ou encadeada.
22. Uma lista pode ser linear ou não linear, dependendo de sua implementação.
23. A raiz de uma árvore é o nó que não possui ancestrais.
24. Árvores binárias de busca são ordenadas.
25. Em média, a busca em uma árvore binária de busca é mais rápida do que a busca em uma lista baseada em matrizes.
26. Em média, a busca em uma árvore binária de busca é mais rápida do que a busca em uma lista.
27. Uma árvore binária de busca é sempre balanceada.
28. Dado o número de nós e o número de níveis em uma árvore de busca binária, você pode determinar a eficiência relativa de uma busca na árvore.
29. A inserção em uma árvore de busca binária é sempre em um nó folha.
30. Uma árvore de busca binária é outra implementação de uma lista ordenada.

O algoritmo a seguir (usado nos Exercícios de 31 a 33) é um laço controlado por contagem que vai de 1 a 5. A cada iteração, o contador do laço é impresso ou colocado em uma pilha dependendo do resultado da função booleana **RanFun()**. (O comportamento de RanFun() é imaterial.) Ao final do laço, os itens na pilha são removidos e impressos. Devido às propriedades lógicas de uma pilha, este algoritmo não pode imprimir certas sequências dos valores do contador do laço. Você recebe uma saída e é perguntado se o algoritmo poderia gerar a saída. Responda da seguinte forma:

A. Verdadeiro B. Falso C. Informações insuficientes.

```
count ← 0
while (count < 5)
   count ← count + 1
   if (RanFun())
      write count
   else
      Push(Pilha, count)
while (not isEmpty(Pilha))
   Pop(Pilha, number)
   write number
```

31. A seguinte saída é possível usando uma pilha: 1 3 5 2 4.
32. A seguinte saída é possível usando uma pilha: 1 3 5 4 2.
33. A seguinte saída é possível usando uma pilha: 1 3 5 1 3.

O algoritmo a seguir (usado nos Exercícios de 34 a 36) é um laço controlado por contagem que vai de 1 a 5. A cada iteração, o contador do laço é impresso ou colocado em uma fila dependendo do resultado da função booleana **RanFun()**. (O comportamento de RanFun() é imaterial.) Ao final do laço, os itens na fila são retirados da fila e impressos. Devido às propriedades lógicas de uma fila, este algoritmo não pode imprimir certas sequências dos valores do contador do laço. Você recebe uma saída e é perguntado se o algoritmo poderia gerar a saída. Responda da seguinte forma:

A. Verdadeiro B. Falso C. Informações insuficientes.

```
count ← 0
while (count < 5)
   count ← count + 1
   if (RanFun())
      write count
   else
      Enqueue(Fila, count)
while (not isEmpty(Fila))
   Dequeue(Fila, number)
   write number
```

34. A seguinte saída é possível usando uma fila: 1 3 5 2 4.
35. A seguinte saída é possível usando uma fila: 1 3 5 4 2.
36. A seguinte saída é possível usando uma fila: 1 3 5 1 3.

Os exercícios 37 a 50 são perguntas de resposta curta.

37. O que é escrito pelo seguinte algoritmo a seguir?

```
Push(Pilha, 5)
Push(Pilha, 4)
Push(Pilha, 4)
item ← Pop(Pilha)
item ← Pop(Pilha)
Push(Pilha, item)
while (not isEmpty(Pilha))
   item ← Pop(Pilha)
   write item
```

38. O que é escrito pelo seguinte algoritmo a seguir?

```
Enqueue(Fila, 5)
Enqueue(Fila, 4)
Enqueue(Fila, 4)
item ← Dequeue(Fila)
item ← Dequeue(Fila)
Enqueue(Fila, item)
while (not isEmpty(Fila))
   item ← Dequeue(Fila)
   write item
```

39. Escreva um algoritmo que iguale uma variável **base** ao último elemento da pilha, deixando a pilha vazia.
40. Escreva um algoritmo que iguale uma variável **base** ao último elemento da pilha, deixando a pilha inalterada.
41. Escreva um algoritmo para criar uma cópia de uma **Pilha**, deixando **Pilha** inalterada.
42. Escreva um algoritmo que iguale **ultimo** ao último elemento de uma fila, deixando a fila vazia.
43. Escreva um algoritmo que iguale **ultimo** ao último elemento de uma fila, deixando a fila inalterada.
44. Escreva um algoritmo para criar uma cópia de uma **Fila**, deixando **Fila** inalterada.
45. Escreva um algoritmo **Replace** que receba uma pilha e dois itens (**item1** e **item2**). Se o primeiro item estiver na pilha, substitua-o pelo segundo item, deixando o restante da pilha inalterado.
46. Escreva um algoritmo **Replace** que receba uma fila e dois itens (**item1** e **item2**). Se o primeiro item estiver na fila, substitua-o pelo segundo item, deixando o restante da fila inalterado.
47. Desenhe a árvore binária de busca cujos elementos são inseridos na seguinte ordem:

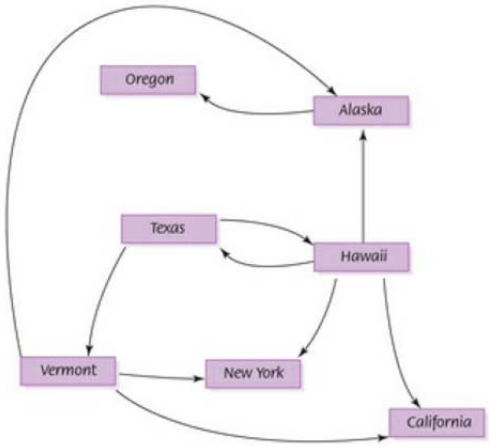
    50, 72, 96, 107, 26, 12, 11, 9, 2, 10, 25, 51, 16, 17 e 95.
48. Se a função **Print** for aplicada à árvore formada no Exercício 47, em que ordem os elementos seriam impressos?
49. Examine o seguinte algoritmo e aplique-o à árvore formada no Exercício 47. Em que ordem os elementos seriam impressos?

```
Print2(tree)
   if (tree != NULL)
      Print(right(tree))    // Chamada recursiva R1
      write info(tree)
      Print(left(tree))     // Chamada recursiva R2
```

50. Examine o seguinte algoritmo e aplique-o à árvore formada no Exercício 47. Em que ordem os elementos seriam impressos?

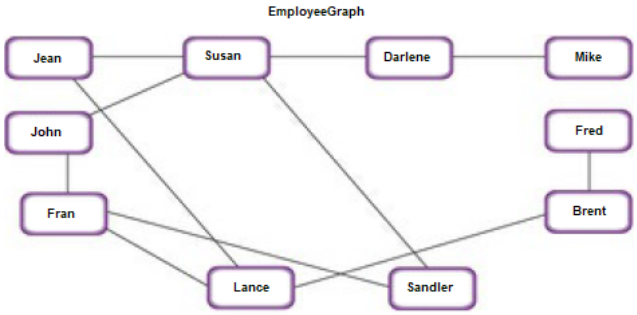
```
Print3(tree)
   if (tree != NULL)
      Print(right(tree))    // Chamada recursiva R1
      Print(left(tree)      // Chamada recursiva R2
      write info(tree)
```

Os Exercícios 51 a 55 são questões de resposta curta baseadas no grafo direcionado apresentado ao lado.



51. Existe um caminho do Oregon para qualquer outro estado no grafo?
52. Existe um caminho do Havaí para todos os outros estados no grafo?
53. De qual(is) estado(s) no grafo existe um caminho para o Havaí?
54. Mostre a tabela que representa este grafo.
55. É possível ir de Vermont ao Havaí?

Os Exercícios 56 a 60 são questões de resposta curta baseadas no grafo direcionado apresentado ao lado.



56. Mostre o percurso da DFS de Jean a Sandler.
57. Mostre o percurso da DFS de Lance a Darlene.
58. Mostre o percurso da BFS de Jean a Sandler.
59. Mostre o percurso da BFS de Lance a Darlene.
60. Mostre a tabela que representa este gráfico.

Os exercícios 61 a 69 são exercícios de resposta curta.

61. Dado o registro **Lista** contendo os valores do array e a variável **length**, escreva o algoritmo para a função **getLength** (obter o comprimento do array).
62. Suponha que o registro Lista tenha uma variável adicional **posicaoAtual**, inicializada com o primeiro item da lista. Qual é o valor inicial de **posicaoAtual**?
63. Escreva o algoritmo para a função **maisItens**, que retorna TRUE se houver mais itens na lista e FALSE caso contrário.
64. Escreva o algoritmo para **getNext(Lista, item)** de forma que o item seja o próximo item na lista. Certifique-se de atualizar **posicaoAtual**.
65. Os exercícios 61 a 64 criam os algoritmos que permitem ao usuário de uma lista ver os itens um de cada vez. Escreva o algoritmo que usa essas operações para imprimir todos os itens de uma lista.
66. O que acontece se uma inserção ou exclusão ocorrer no meio de uma iteração pela lista? Explique.
67. Você consegue pensar em uma maneira de impedir que o usuário faça uma inserção ou exclusão durante uma iteração?
68. Diferencia entre parâmetros de valor e parâmetros de referência.
69. Como argumentos e parâmetros são correspondidos?

QUESTÕES DE REFLEXÃO

1. Uma planilha é uma tabela com linhas e colunas. Pense em um TAD Planilha. Quais operações você precisaria para construir a tabela? Quais operações você precisaria para manipular os valores na tabela?
2. Árvores binárias, árvores de busca binárias e grafos são visualizados como nós e setas (ponteiros) que representam os relacionamentos entre os nós. Compare essas estruturas em termos das operações permitidas. Uma lista pode ser uma árvore? Uma árvore pode ser uma lista? Uma árvore pode ser um grafo? Um grafo pode ser uma árvore? Como as estruturas se relacionam entre si?
3. Antes dos computadores, as conversas de bate-papo eram consideradas privadas. Como a tecnologia da computação mudou essa premissa?
4. Como os direitos dos funcionários colidem com os direitos de privacidade no local de trabalho?